

算法设计与分析

北郊 HH

2025 年 5 月 11 日

目录

1	chapter10: 随机算法	1
1.1	导论	1
1.2	随机算法举例	2
1.2.1	随机快速排序	2
1.2.2	随机选择	3

1 chapter10: 随机算法

1.1 导论

1. 与确定性算法相比, 随机算法所需运行时间或空间通常较小;
2. 随机算法易于理解和实现。

1. Las Vegas 算法¹: 总是给出正确的解, 或者无解

通常需要分析算法的平均运行时间, 运行时间本身是一个随机变量。

期望运行时间是输入规模的多项式且总能给出正确答案的随机算法称为有效的拉斯维加斯算法, 也称为 **ZPP** 类算法 (zero-error probabilistic polynomial time) 算法, 即零错误概率多项式时间随机算法。

2. Monte Carlo 算法: 总是给出解, 偶尔可能是错误解, 可以通过多次运行原算法, 且满足每次运行时随机选择都相互独立, 使产生错误解的概率减到任意小。

通常需要分析算法的出错概率, 总是在多项式时间内运行且出错概率不超过 $1/3$ 的随机算法称为有效的 * 蒙特卡罗型随机算法, 也称为 **BPP** 类算法 (bounded-error probabilistic polynomial time), 即错误概率有限的多项式时间随机算法。

错误类型:

- 弃真型错误: 在求解一个判定问题时把本应该接受的输入误判为拒绝。
- 取伪型错误: 就是把本应该拒绝的输入误判为接受。

¹Las Vegas 和 Monte Carlo 不是一种算法的名称, 而是对一类随机算法的特性的概括。不太可能为 **NP** 完全问题设计出多项式时间的随机算法

根据是否同时出现两类错误，蒙特卡洛型算法又分为：

- (a) **单侧错误的蒙特卡洛型算法**：在所有输入上，要么犯弃真的错误，要么犯取伪的错误，不同时出现两种不同的错误。
- 有效的**弃真型**单侧错误随机算法称为 **RP 类算法** (Randomized Polynomial-time)
 - 有效的**取伪型**单侧错误随机算法称为 **coRP 类算法** (Complementary Randomized Polynomial-time, RP 的补类)
- (b) **双侧错误的蒙特卡洛型算法**：在所有输入上可以同时出现两种不同的错误。

对于随机算法 A ，其在大小为 n 的一个实例 I 上的运行时间可能因不同次执行而异。因此，更自然的性能度量是 A 在实例 I 上的期望运行时间，也即算法 A 反复求解实例 I 所花费的平均时间。

- 对于确定算法，求解期望运行时间时，概率分布定义在**输入**上
- 对于随机算法，求解期望运行时间时，概率分布定义在**算法**上

期望时间：对于大小为 n 的特定输入 I ，算法的期望时间就是在随机数的不同序列上的时间的平均：

$$ET(A, I) = \sum_{\text{random sequence } R} \text{probability}(R) * \text{time}(A, I, R)$$

最坏情况下的期望时间：

$$W_ET(A) = \max_{\text{input } I \text{ with size } n} (\text{random sequence } R \text{ probability}(R) * \text{time}(A, I, R))$$

1.2 随机算法举例

1.2.1 随机快速排序

其实是在快速排序的基础上，增加了**随机选取**的步骤。

Input : An array $A[1 \dots n]$ of n elements
Output: The elements in A sorted in nondecreasing order

```
1 quicksort (A,low,high)
2 if low < high then
3   | SPLIT(A [low...high ], w)
4   | // w is the new position of A [low ]
5   | quicksort (A,low, w-1)
6   | quicksort (A, w +1, high)
7 end
```

Algorithm 1: Quicksort

该算法在最坏情况的运行时间仍然是 $\Theta(n^2)$ ，但这个最坏情况不是由于输入形式，而是由于随机数生成器每次非常不幸地选择了不好的划分元素，这种情况不大可能发生。算法的期望运行时间为 $\Theta(n \log n)$ ，是一个 ZPP 类算法

Input : An array A of n elements

Output: The elements in A sorted in nondecreasing order

```
1 rquicksort (A,low,high)
2 if low < high then
3   v ← random (low, high) interchange A [low ] and A [v ]
4   SPLIT (A [low...high ], w, v)
5   rquicksort (A,low, w-1)
6   rquicksort (A,w +1, high)
7 end
```

Algorithm 2: Randomized Quicksort

1.2.2 随机选择